

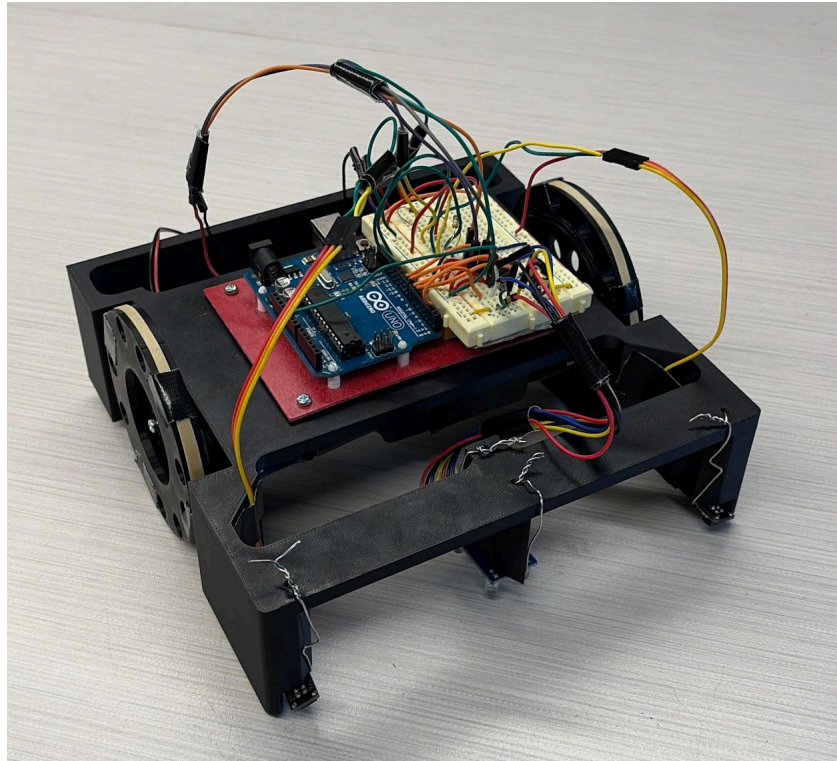
Team #20

Greg Svensson - gas273

Jake Dick - jdd249

Hugo Mazzali - hlm89

1. Robot Design and Strategy Overview



Our robot was designed to collect as many cubes as possible while still remaining simple and reliable. The robot's chassis was 3D printed with a top mounted arduino to maximize cube collection area underneath. We also 3D printed larger wheels to reach cubes faster than our opponents. Chassis and wheels CAD models are included in Appendix C.

The electrical system consisted of two DC motors controlled by an H-bridge, a front-mounted color sensor, and two QTI sensors mounted on the front left and right sides of the robot. The color sensor detected changes in the arena floor color, while the QTI sensors detected the black border to keep the robot in bounds. The full circuit diagram is included in Appendix B.

The software used a simple strategy focused on sweeping through the center of the board where most cubes were located. The robot would drive forward until it detects a color change, then turn 90 degrees and continue driving. If either QTI sensor detected a borderline, the robot turned around and kept moving. Multiple sensor readings were averaged to improve reliability and reduce false detections. A flowchart of the robot

behavior and the fully commented Arduino code are included in Appendix D and Appendix E.

2. Design Process Reflection

Our group went through two main design iterations before arriving at the final robot. At the beginning of the project, we planned to build a cube collection system similar to a tennis ball pickup mechanism using a rolling wire cage. However, after beginning prototyping, we realized the design was much more complicated than expected. We had already purchased wire and spent time testing ideas, so changing directions was difficult, but we decided to simplify the design.

Next, we shifted toward a simpler and more reliable strategy focused on sweeping cubes rather than rolling over them to pick them up. To complete the milestones, we used the provided chassis and prototyped it out of cardboard. For our final iteration, we designed our own 3D printed chassis and wheels, which can be seen in Appendix C. Completing the milestones with the provided chassis gave us time to test and iterate before committing to our final design. One challenge we faced was making the robot consistently detect color changes as the sensor ground distance varied, resulting in drastically different outputs. We improved this on the final design by mounting the color sensor more rigidly via a safety wire on a flatter surface, much closer to the ground, and tuning our code to be more sensitive. Overall, our group progressed quickly by working through ideas and being willing to change designs when things didn't work out.

3. Competition Analysis

Overall, our robot performed very well on competition day. We placed first in our round robin bracket and advanced to the single elimination round, where we lost in the first match. The robot's biggest strength was collecting a large number of cubes quickly at the start of matches. The wide front area and larger wheels worked very effectively at getting our bot to collect the first cubes in the center of the board very quickly.

Our biggest weakness was head-to-head contact with other robots. In two of our losses, our robot drove directly into the opposing robot, and both bots kept pushing forward, allowing the other robot to push us backward and take our cubes.

4. Conclusions

Overall, our project was successful because we focused on keeping the robot simple and easy to test. Although our original design ideas were more complicated, simplifying the robot early on helped us make faster progress and produce a better-performing final design. If we were to do the project again, we would improve the robot's interactions with other robots. This could be as simple as adding one line of

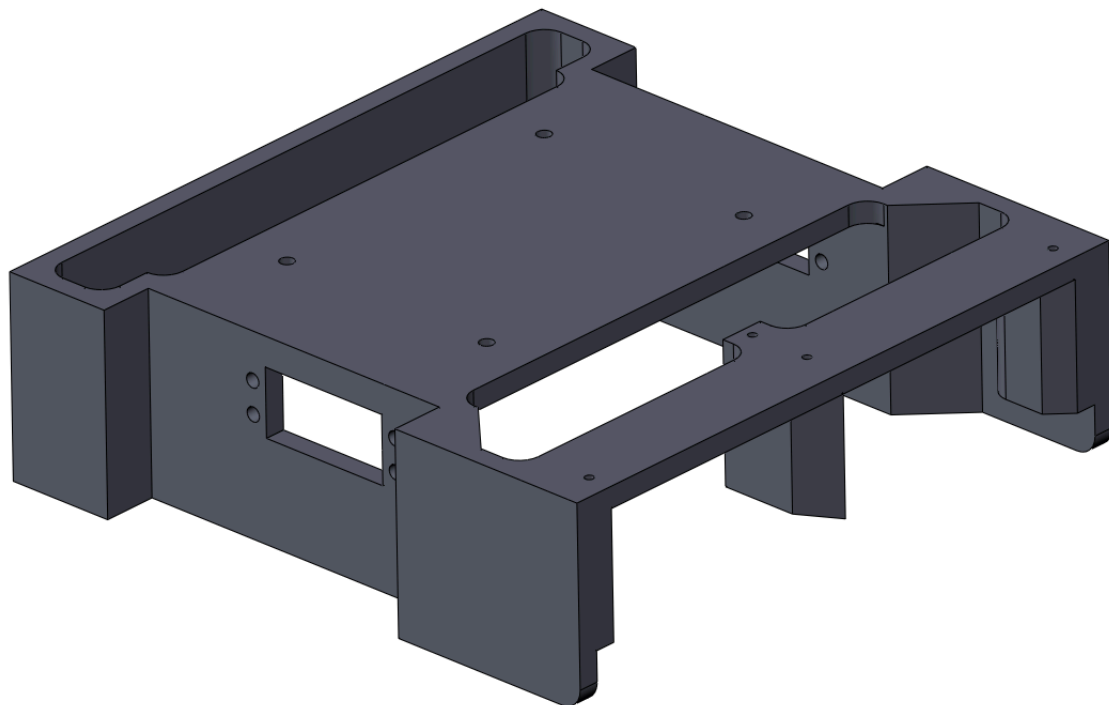
code to turn around after driving forward for too long without making progress in order to address our weakness, or just make it much heavier to prevent being pushed as easily. We'd also be interested in looking into the ultrasonic sensor as another possibility to address this weakness.

Our biggest advice for future students would be to prototype and test early, avoid overcomplicating the design, and focus on reliability over adding too many features. From our observations on competition day, the simple designs that consistently worked performed better than the complex designs.

Appendix A - BOM

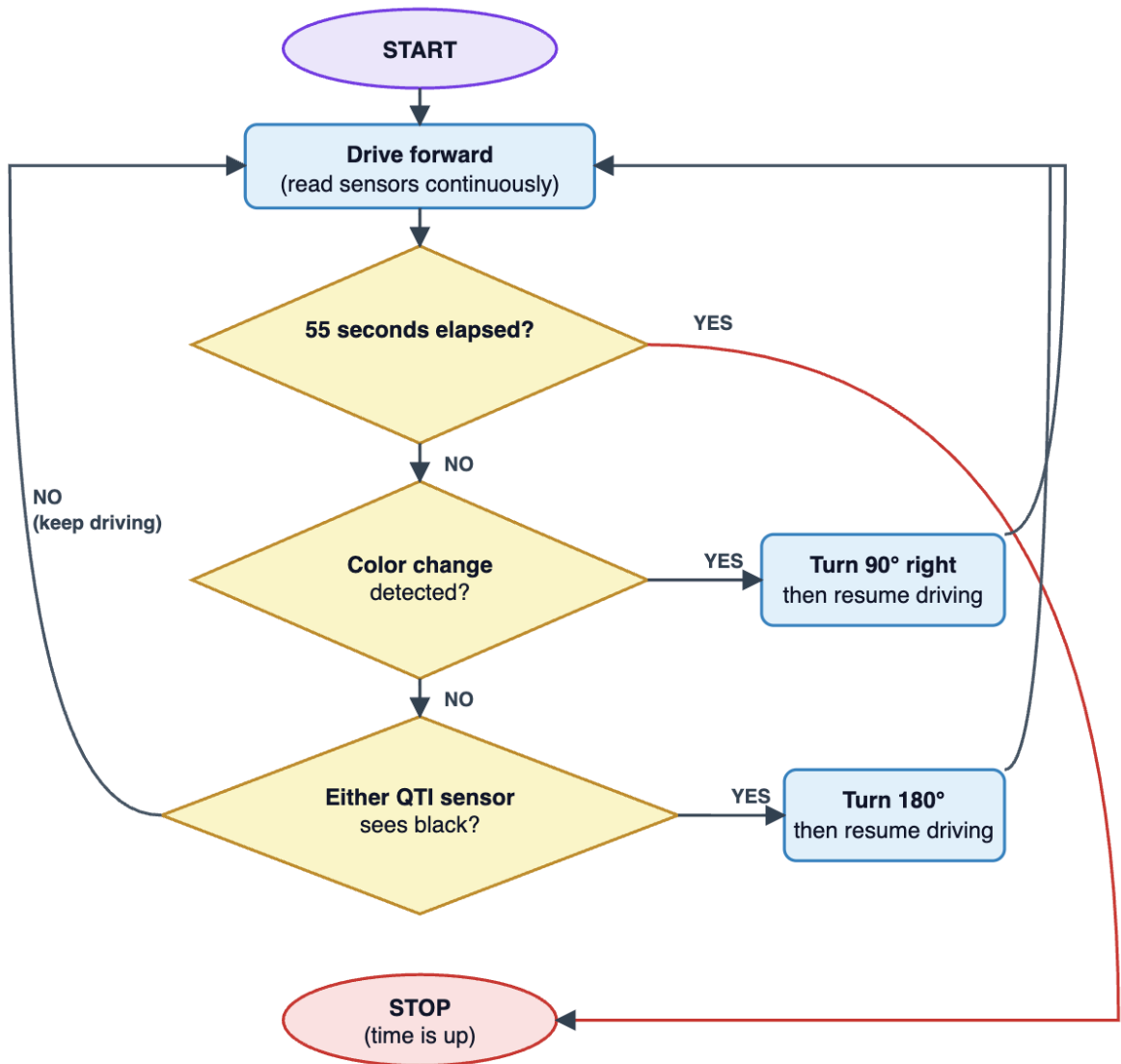
Bill of Materials							
Category	Purchasing Link	Application	Quantity	Additional Information	Mass (g)	Unit Price (\$)	Supplier
0.067" Wire	https://www.mcmaster.com/9495K94/	Fastening and structure	1	21'		9.9	McMaster-Carr
0.041" Wire	https://www.mcmaster.com/8908K32/	Fastening and structure	1	55'		8.85	McMaster-Carr
3d print	https://docs.google.com/forms/d/e/1FAIpQLSdJQnPpIStMpOAXD3isLj1It_DL_eBmYtF3ZyswIIQIFQq8A/viewform	Wheels	2	25.56382 cubic cm (10% infill and PLA material)	3.195	2.278	Rapid Prototyping Lab
3d print	https://docs.google.com/forms/d/e/1FAIpQLSdJQnPpIStMpOAXD3isLj1It_DL_eBmYtF3ZyswIIQIFQq8A/viewform	Chassis	1	420.79 cubic cm (7% infill and PLA material)	36.819	15.7276	Rapid Prototyping Lab
Total (\$)							36.7556

Appendix C - CAD Files & Drawings





Appendix D – Flowchart



Appendix E – Arduino Code

```
/*
Color sensor timing variables.
These are volatile because they are
changed inside the interrupt.
*/
volatile float period = 0;
volatile int timer = 0;

/*
Tuned turn timing values.
*/
#define TURN_90_TIME 540
#define TURN_180_TIME 900

/*
After the first 90 degree turn, the robot
runs for this long.
*/
#define POST_90_RUN_TIME_MS 55000

/*
Blue averages around 67 us.
Yellow averages around 87 us.
The midpoint, 77 us, is used to separate
blue from yellow.
*/
#define BLUE_AVG 67
#define YELLOW_AVG 87
#define BLUE_YELLOW_SPLIT 77

/*
Color change must be confirmed by
several consecutive readings
before the robot turns.
*/
#define COLOR_CONFIRM_SAMPLES 10
#define
COLOR_CONFIRM_EXTRA_DELAY_MS
15

// QTI sensor pins
#define LEFT_QTI PB4 // D12

#define RIGHT_QTI PB5 // D13

/*
Interrupt routine for measuring the
color sensor pulse width.
*/
ISR(PCINT2_vect) {
    if (PIND & (1 << PIND2)) {
        TCNT1 = 0;
    }
    else {
        timer = TCNT1;
        period = timer;
    }
}

/*
Sets up the color sensor input and
Timer1.
*/
void initColor() {
    DDRD &= ~(1 << DDD2);

    TCCR1A = 0b00000000;
    TCCR1B = 0b00000001;

    sei();
}

/*
Takes one color sensor reading.
*/
int getColor() {
    PCICR |= (1 << PCIE2);
    PCMSK2 |= (1 << PCINT18);

    _delay_ms(10);

    PCICR &= ~(1 << PCIE2);
    PCMSK2 &= ~(1 << PCINT18);
}
```



```

    int period_us = (int)(period * 2 *
0.0625);
    return period_us;
}

/*
Averages 5 color readings to reduce
noise.
*/
int getStableColor() {
    int total = 0;

    for (int i = 0; i < 5; i++) {
        total += getColor();
        _delay_ms(10);
    }

    return total / 5;
}

/* ----- COLOR CLASSIFICATION
----- */

/*
Returns 1 if the reading is classified as
yellow.
Returns 0 if the reading is classified as
blue.
*/
int colorIsYellow(int colorReading) {
    if (colorReading >=
BLUE_YELLOW_SPLIT) {
        return 1;
    }
    else {
        return 0;
    }
}

/*
Checks whether the current color is
different from the starting color.
*/

```

```

int colorChanged(int currentColor, int
startColor) {
    int currentIsYellow =
colorIsYellow(currentColor);
    int startIsYellow =
colorIsYellow(startColor);

    if (currentIsYellow != startIsYellow) {
        return 1;
    }

    return 0;
}

/*
Requires the color change to remain
stable for several readings.
*/
int confirmColorChange(int startColor) {
    for (int i = 0; i <
COLOR_CONFIRM_SAMPLES; i++) { //
Make sure there are
COLOR_CONFIRM_SAMPLES # of
consecutive color readings
        int reading = getColor();

        if (!colorChanged(reading, startColor))
        {
            return 0;
        }

        for (int j = 0; j <
COLOR_CONFIRM_EXTRA_DELAY_MS;
j++) {
            _delay_ms(1);
        }
    }

    return 1;
}

/* ----- QTI EDGE SENSORS ----- */

/*

```

```

    Sets QTI pins as inputs.
    */
    void initQTI() {
        DDRB &= ~((1 << LEFT_QTI) | (1 <<
            RIGHT_QTI));
    }

    /*
    Returns 1 if the left QTI sees black.
    */
    int leftQTIBlack() {
        return (PINB & (1 << LEFT_QTI)) ? 1 : 0;
    }

    /*
    Returns 1 if the right QTI sees black.
    */
    int rightQTIBlack() {
        return (PINB & (1 << RIGHT_QTI)) ? 1 : 0;
    }

    /*
    Returns 1 if either QTI sensor sees
    black.
    */
    int anyQTIBlack() {
        int leftBlack = leftQTIBlack();
        int rightBlack = rightQTIBlack();

        if (leftBlack || rightBlack) {
            return 1;
        }

        return 0;
    }

    /* ----- MOTOR CONTROL ----- */

    /*
    Stops both motors.
    */
    void stopMotors() {
        PORTD &= ~((1 << PORTD6) | (1 <<
            PORTD7));

```

```

        PORTB &= ~((1 << PORTB0) | (1 <<
            PORTB1));
    }

    /*
    Drives both motors forward.
    */
    void driveForward() {
        // Left motor forward
        PORTD |= (1 << PORTD6);
        PORTD &= ~(1 << PORTD7);

        // Right motor forward
        PORTB |= (1 << PORTB1);
        PORTB &= ~(1 << PORTB0);
    }

    /*
    Delay wrapper.
    */
    void delayMs(int ms) {
        for (int i = 0; i < ms; i++) {
            _delay_ms(1);
        }
    }

    /*
    Turns the robot right 90 degrees.
    */
    void turnRight90() {
        // Left motor forward
        PORTD |= (1 << PORTD6);
        PORTD &= ~(1 << PORTD7);

        // Right motor backward
        PORTB &= ~(1 << PORTB1);
        PORTB |= (1 << PORTB0);

        delayMs(TURN_90_TIME);

        stopMotors();
    }

    /*

```

```

    Turns the robot right 180 degrees.
    */
    void turnRight180() {
        // Left motor forward
        PORTD |= (1 << PORTD6);
        PORTD &= ~(1 << PORTD7);

        // Right motor backward
        PORTB &= ~(1 << PORTB1);
        PORTB |= (1 << PORTB0);

        delayMs(TURN_180_TIME);

        stopMotors();
    }

    /* ----- MAIN PROGRAM ----- */

    int main(void) {
        // Left motor outputs: D6, D7
        DDRD |= (1 << DDD6) | (1 << DDD7);

        // Right motor outputs: D8, D9
        DDRB |= (1 << DDB0) | (1 << DDB1);

        initColor();
        initQTI();

        stopMotors();
        delayMs(1000);

        int startColor = getStableColor();
        int hasTurned90 = 0;
        long post90ElapsedMs = 0;

        driveForward();

        while (1) {

            /*
                Phase 1:
                Drive forward until a stable
                blue/yellow color change is detected.
                Then turn right 90 degrees once.

```

```

            */
            if (!hasTurned90) {
                int currentColor = getStableColor();

                if (colorChanged(currentColor,
                                startColor)) {
                    if (confirmColorChange(startColor))
                    {
                        stopMotors();
                        delayMs(200);

                        turnRight90();

                        hasTurned90 = 1;
                        post90ElapsedMs = 0;

                        delayMs(200);
                        driveForward();
                    }
                }

                delayMs(50);
            }

            /*
                Phase 2:
                After the 90 degree turn, drive for 55
                seconds.
                If either QTI detects black, turn 180
                degrees and keep driving.
            */
            else {
                if (post90ElapsedMs >=
                    POST_90_RUN_TIME_MS) {
                    stopMotors();

                    while (1) {
                        stopMotors();
                    }
                }

                if (anyQTIBlack()) {
                    stopMotors();
                    delayMs(200);

```

```
    post90ElapsedMs += 200;

    turnRight180();
    post90ElapsedMs +=
TURN_180_TIME;

    delayMs(200);
    post90ElapsedMs += 200;

    driveForward();

    /*
    Move forward briefly after turning
    so the QTI sensors
```

get off the black line and do not
immediately trigger again.

```
    */
    delayMs(300);
    post90ElapsedMs += 300;
}

    driveForward();

    delayMs(50);
    post90ElapsedMs += 50;
}
}
}
```